Gharpayy CRM MVP
System Design Document

By,

Khriesezo Peseyie

To be submitted to,

Gharpayy Hiring Management

For,

Gharpayy CRM MVP Assignment -
Build a Lead Management System (CRM)

Github Username: **Areaxis**
https://github.com/Areaxis/gharpayy-crm-mvp

Project Overview

This document describes the architecture, database design, and scalability considerations for the Gharpayy CRM MVP.

The goal of the system is to manage property leads efficiently by capturing inquiries, assigning them to agents, tracking their progress through a sales pipeline, and scheduling property visits.

The system is designed to be simple enough for rapid development while remaining scalable enough to evolve into a full production CRM platform.

Table of Content

1. Why I Am a Good Fit?

2. System Architecture

3. Database Design

4. Lead Lifecycle

5. Lead Assignment Strategy

6. Dashboard Analytics

7. Scalability for Production

8. Technical Expectations

1. Why I Am a Good Fit for Building This System?

The objective of this assignment is to design and implement a lightweight CRM capable of managing property leads effectively. My approach focuses on creating systems that are both practical for immediate use and structured for long-term scalability.

Key strengths of this implementation include:

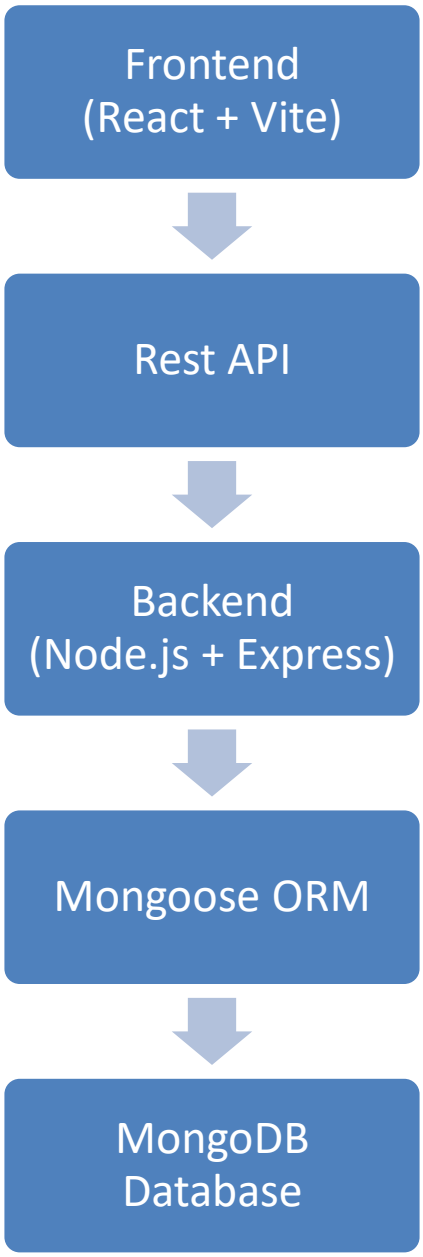
- Clear separation between frontend, backend, and database layers
- Modular backend architecture for maintainability
- Feature-based frontend architecture for scalability
- Automated lead assignment to distribute workload fairly
- Lead activity timeline for transparency and auditing
- Visit scheduling with automated pipeline updates
- Dashboard analytics for monitoring lead performance

This structure allows the MVP to demonstrate a complete lead management workflow while remaining flexible for future expansion.

2. System Architecture

The system follows a three-tier architecture, consisting of a frontend application, backend API server, and database layer.

a. Architecture Overview



b. Frontend Layer

The frontend application is responsible for user interaction and visualization of CRM data.

Technologies used:

• React

• Vite

• TailwindCSS

• React Router

Responsibilities of the frontend include:

• Lead creation form

• Pipeline visualization

• Visit scheduling interface

• Dashboard analytics display

The frontend uses a feature-based architecture, organizing code by domain features rather than by file type.

Features Include:

- This approach improves scalability and maintainability as the application grows.

c. Backend Layer

The backend provides REST API endpoints that manage business logic and database interactions.

Technologies used:

• Node.js

• Express

• MongoDB

• Mongoose

Backend responsibilities include:

• Creating and updating leads

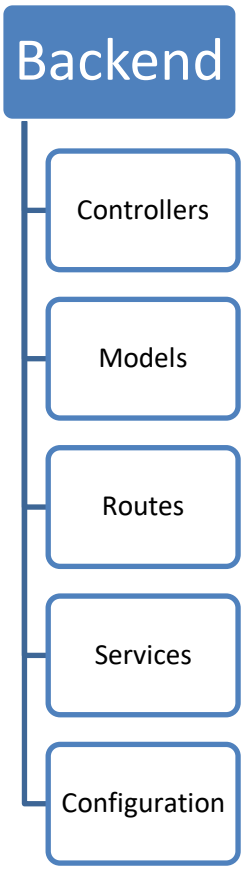
• Automatically assigning leads to agents

• Scheduling property visits

• Updating pipeline status

• Running background reminder services

Backend Structure:



- This separation ensures better code organization and simplifies future maintenance.

Background Services:

The system includes a background reminder system implemented using Node Cron.

The service periodically checks for inactive leads where:

```
lastActivity > 24hrs
```

When inactivity is detected, the system flags the lead for follow-up to ensure no potential customer is overlooked.

3. Database Design

The application uses MongoDB, a document-based NoSQL database that allows flexible data modeling and easy schema evolution.

Three primary collections are used in the system:

a. Leads Collection

The Leads collection stores customer inquiries and tracks their progress through the sales pipeline.

Fields include:

Leads Collection
• name • phone • source • location • moveInDate • gender • occupation • score • status • assignedAgent • lastActivity • activity timeline • createdAt

The activity field stores an array of actions taken on the lead, such as:

Activity Field
• Lead created • Assigned to agent • Visit scheduled • Status updated

- This allows the system to maintain a complete history of interactions with each lead.

b. Agents Collection

The Agents collection stores information about sales agents responsible for handling leads.

Fields include:

Agents Collection
• name • email • activeLeads

- The activeLeads field helps distribute leads evenly among agents using workload balancing.

c. Visits Collection

The Visits collection stores property visit appointments.

Fields include:

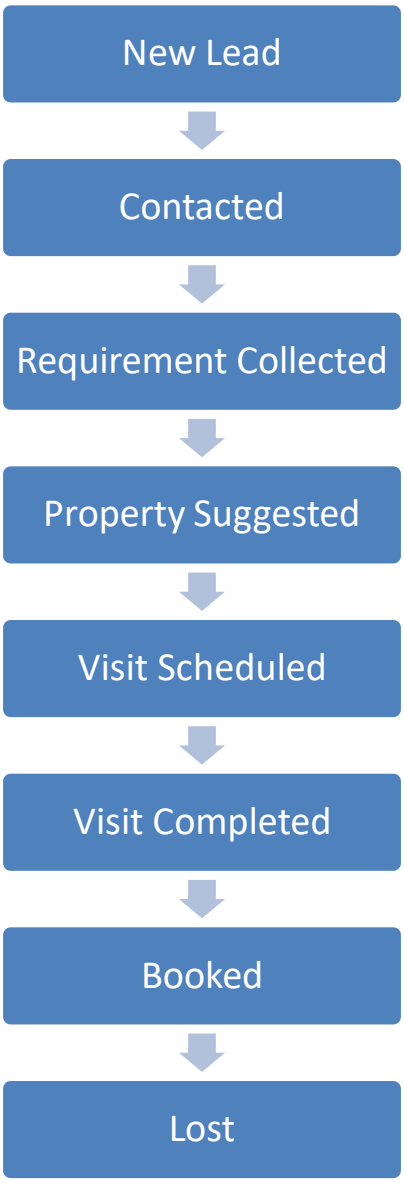
Visits Collection
• leadId • property • date • outcome • createdAt

- The outcome of a visit automatically updates the lead's pipeline stage.

4. Lead Lifecycle

Leads progress through defined stages within the CRM pipeline.

The stages include:



- Each stage change is recorded in the lead's activity timeline, ensuring a clear record of all interactions.

5. Lead Assignment Strategy

The system automatically assigns new leads to agents based on workload balancing.

The algorithm selects the agent with the fewest active leads.

Example logic:

```
Agent.findOne().sort({ activeLeads: 1 })
```

Benefits of this approach:

- Fair distribution of work
- Faster response times
- Clear accountability for each lead

6. Dashboard Analytics

The dashboard provides real-time insights into the performance of the sales process.

Displayed metrics include:

Dashboard Analysis
• Total leads • Visits scheduled • Conversion rate • Agent performance leaderboard • Recent lead activity

- This allows managers to quickly assess team performance and identify opportunities for improvement.

7. Scalability for Production

While the MVP is designed for smaller teams, several improvements can support large-scale production usage.

a. Infrastructure

The system could be deployed using cloud platforms such as:

• AWS

• Google Cloud Platform

• DigitalOcean

Key infrastructure components would include:

• Load balancers

• Containerized services

• Managed databases

b. Background Job Processing

Instead of using simple cron jobs, production systems should use distributed job queues such as Redis and BullMQ.

Advantages include:

• Reliable task execution

• Automatic retries

• Scalable worker processes

c. Database Scaling

MongoDB supports horizontal scaling using:

• Replica sets for high availability

• Sharding for distributing large datasets

These techniques allow the system to support large numbers of leads and agents.

d. Performance Optimization

Performance improvements could include:

• API caching

• Query indexing

• Pagination for large datasets

Indexes would typically be added on:

• lead status

• assignedAgent

• createdAt

8. Technical Expectations and Tools Required

To build a full production version of this CRM, the following tools and infrastructure would be recommended.

Development:

• React

• Node.js

• Express

• MongoDB

• TailwindCSS

Infrastructure:

• Docker

• Cloud hosting (AWS or GCP)

• CI/CD pipelines

Messaging and Notifications:

• WebSockets

• Firebase notifications

Monitoring:

• Sentry for error tracking

• Prometheus for metrics

• Grafana for visualization

Conclusion

This MVP demonstrates a working lead management CRM capable of capturing leads, assigning them to agents, scheduling property visits, and tracking conversion performance.

The architecture is intentionally modular and scalable, allowing the system to evolve into a full production CRM platform with additional integrations and infrastructure improvements.